arXiv:2402.13144v3 [cs.LG] 30 Dec 2024

Neural Network Diffusion

Kai Wang[1] Dongwen Tang[1] Boya Zeng[2] Yida Yin[3] Zhaopan Xu[1]
Yukun Zhou[1] Zelin Zang[1] Trevor Darrell[3] Zhuang Liu[4*] Yang You[1*]
[1]National University of Singapore [2]University of Pennsylvania
[3]University of California, Berkeley [4]Meta FAIR

Abstract

Diffusion models have achieved remarkable success in image and video generation. In this work, we demonstrate that diffusion models can also generate high-performing neural network parameters. Our approach is simple, utilizing an autoencoder and a diffusion model. The autoencoder extracts latent representations of a subset of the trained neural network parameters. Next, a diffusion model is trained to synthesize these latent representations from random noise. This model then generates new representations, which are passed through the autoencoder's decoder to produce new subsets of high-performing network parameters. Across various architectures and datasets, our approach consistently generates models with comparable or improved performance over trained networks, with minimal additional cost. Notably, we empirically find that the generated models are not memorizing the trained ones. Our results encourage more exploration into the versatile use of diffusion models. Our code is available here.

1. Introduction

The origin of diffusion models can be traced back to non-equilibrium thermodynamics [29, 64]. Diffusion was first applied to progressively remove noise from inputs and generate clear images in [64]. Later works, such as DDPM [26] and DDIM [66], refined diffusion models with a training paradigm characterized by forward and reverse processes.

At that time, diffusion models were not yet capable of generating high-quality images. Guided-Diffusion [13] enhanced diffusion models with a more effective architecture and thereby surpassed GAN-based methods [28, 77]. Since then, GLIDE [47], Imagen [59], DALL·E 2 [52], and Stable Diffusion [57] have achieved photorealistic image generations, becoming widely adopted by artists and creatives.

*equal advising

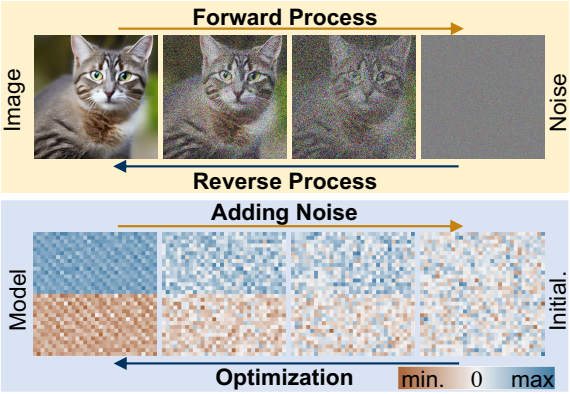


Figure 1. *Top* illustrates the standard diffusion process in image generation. *Bottom* shows the parameter heatmap of the batch normalization (BN) layer at various stages of ResNet-18 training on CIFAR-100. In the heatmap, *the upper half* is BN weights, while *the lower half* is BN biases. Color corresponds to parameter value.

Despite the great success of diffusion models in visual generation, their potential in other domains remains largely unexplored. In this work, we demonstrate the surprising capability of diffusion models in *generating high-performing neural network parameters*. Parameter generation aims to produce neural network parameters that perform well on specific tasks and has been approached through prior and probability modeling, such as stochastic neural network [44, 60, 65, 72] and Bayesian neural network [31, 32, 46, 56]. However, leveraging diffusion models for parameter generation has not been well-explored yet.

Let's take a closer look at neural network training and diffusion-based image generation in Figure 1. They share two commonalities: i) Both neural network training and the reverse process of diffusion models transition from random noise or initialization to specific distributions. ii) High-quality images and well-optimized parameters can likewise be degraded into simple distributions, such as Gaussian distribution, through multiple rounds of noise addition.

Based on the observations above, we introduce a novel

approach for parameter generation, named Neural Network Diffusion (**p-diff**, where 'p' stands for parameter), which employs a standard latent diffusion model to synthesize new sets of parameters. Our method is simple, comprising an autoencoder and a standard latent diffusion model to learn the distribution of high-performing parameters. First, we train the autoencoder on a subset of neural network parameters to extract their latent representations. The diffusion model is then optimized to generate these latent representations from random noise. This allows us to synthesize new latent representations from random noise and feed these representations through the trained autoencoder's decoder to produce new, high-performing model parameters.

Our approach has the following characteristics: i) It consistently achieves similar or enhanced performance compared to the training data (*i.e.,* models trained by gradient-based optimizers) across various datasets and architectures. ii) Our generated models show great differences in predictions compared to the models in training data. Therefore, our approach synthesizes novel, high-performing model parameters rather than memorizing the training samples. We hope our work can motivate further research to expand the applications of diffusion models to other domains.

2. Neural Network Diffusion

In this section, we review the preliminaries of diffusion models and introduce the components of our approach.

2.1. Preliminaries of Diffusion Models

Diffusion models typically consist of forward and reverse processes in a multi-step chain indexed by timesteps. We introduce these two processes in the following:

Forward process. Given a sample $x_0 \sim q(x)$, the forward process progressively adds Gaussian noise for T steps and obtain $x_1, x_2, \cdots, x_T$. The process is formulated as:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t \mathbf{I}), \tag{1}$$

where $\mathcal{N}$ is a Gaussian distribution, and β_t controls the noise variance at each step t, and $\mathbf{I}$ is the identity matrix. Note, for any arbitrary step t, we can directly sample:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon_t, \tag{2}$$

where $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$ and $\epsilon_t \sim \mathcal{N}(0, \mathbf{I})$.

Reverse process. In contrast to the forward process, the reverse process aims to remove the noise from the input x_t. A neural network, parameterized by θ, is trained to learn this reverse process:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)), \tag{3}$$

where $\mu_\theta(x_t, t)$ and $\Sigma_\theta(x_t, t)$ are the estimated Gaussian mean and variance outputted by the denoising network.

Training and inference. The model p_θ is trained with the variational lower bound [31] of the log-likelihood of x_0. This training objective is expressed as:

$$\mathcal{L}(\theta) = \sum_t \mathcal{D}_{\mathrm{KL}}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t)), \tag{4}$$

where the $\mathcal{D}_{\mathrm{KL}}(\cdot||\cdot)$ denotes the Kullback–Leibler (KL) divergence between two distributions.

During the inference, we first sample a random Gaussian noise $x_t \sim \mathcal{N}(0, \mathbf{I})$ and then obtain $x_{t-1} \sim p_\theta(x_{t-1}|x_t)$ recursively until we get a clean output.

2.2. Overview

We propose neural network diffusion (p-diff) to generate high-performing neural network parameters from random noise. As illustrated in Figure 2, our method consists of two processes: parameter autoencoder and parameter generation. Given a set of high-performing model checkpoints, we first select a particular subset of parameters from every model and flatten them into 1-dimensional vectors. Subsequently, we use an encoder to extract latent representations from these vectors, and a decoder to reconstruct the parameters from the latent representations. Then, a diffusion model is trained to synthesize latent representations from random noise. After training, we use p-diff to generate new parameters via the following chain: random noise $\rightarrow$ reverse process $\rightarrow$ trained decoder $\rightarrow$ generated parameters.

2.3. Parameter Autoencoder

Data preparation. To collect the training data for the autoencoder, we train a model on the target task until convergence and save a checkpoint at each of 300 additional training steps. In our paper, we default to synthesizing a subset of model parameters. Thus, in the 300 additional training steps, we only update the selected subset of parameters via a gradient-based optimizer and fix the remaining parameters of the model. The saved subsets of parameters $S = [s_1, \ldots, s_k, \ldots, s_K]$ are then used to train the autoencoder, where K is the number of the training samples.

Training. We flatten the subset of parameters s_k into 1-dimensional vector $v_k \in \mathbb{R}^d$, where d is the size of the selected subset of parameters. After that, an autoencoder is trained to reconstruct these parameters. To enhance the robustness and generalization of the autoencoder, we add Gaussian noise to both the parameter vector v_k and the latent representation z_k as augmentation, and control the noise scale with σ_v^2 and σ_z^2. Denote the noise on parameter vector as ξ_{v_k} and the noise on latent representation as ξ_{z_k}. The entire encoding and decoding proceses are:

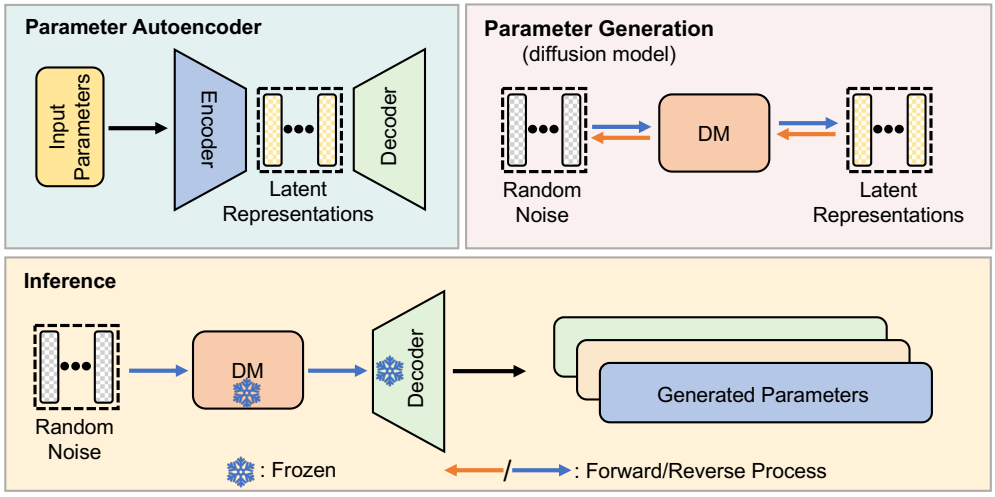


Figure 2. Our approach consists of two processes: parameter autoencoder and parameter generation. Parameter autoencoder aims to extract the latent representations and reconstruct model parameters via the decoder. The extracted representations are used to train a diffusion model (DM). During inference, a random noise vector is fed into the DM and the trained decoder to generate new parameters.

$$
\begin{aligned}
z_k &= \underbrace{f_\phi(v_k + \xi_{v_k})}_{\text{encoding}}, \xi_{v_k} \sim \mathcal{N}(0, \sigma_v^2 \mathbf{I}); \\
\widehat{v_k} &= \underbrace{f_\pi(z_k + \xi_{z_k})}_{\text{decoding}}, \xi_{z_k} \sim \mathcal{N}(0, \sigma_z^2 \mathbf{I}),
\end{aligned} \tag{5}
$$

where $f_\phi(\cdot)$ and $f_\pi(\cdot)$ denote the encoder and decoder parameterized by ϕ and π, respectively, and $\widehat{v_k}$ is the reconstructed output from the decoder. To train our parameter autoencoder, we use the standard objective of minimizing the L^2 norm between $\widehat{v_k}$ and v_k.

2.4. Parameter Generation

Training and inference. One direct strategy to approach parameter generation is to train a diffusion model directly on parameter space. However, the memory cost of this is too heavy, especially when the dimension of the parameters to generate is very large. Therefore, we apply the diffusion process to the latent representations instead. Based on the latent representation z_k extracted from our parameter autoencoder, we can apply the reparamterization trick proposed in DDPM [26] to rewrite Equation 4. Then the training objective for the denoising network simplifies to a mean-squared error:

$$
\mathcal{L}(\theta) = ||\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} z_k + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)||^2, \tag{6}
$$

where $\epsilon_\theta(\sqrt{\bar{\alpha}_t} z_k + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)$ is the predicted noise from the network p_θ and ϵ_t is the ground truth sampled Gaussian noise. Once the noise predictor p_θ has been trained, we directly feed random noise into the reverse process and the trained decoder to generate a new set of high-performing parameters. During evaluation, these generated parameters are concatenated with the remaining parameters in the original model to form a new one.

Design space. Neural network parameters differ from image pixels in several ways, including data type, dimensions, range, and physical interpretation. Unlike images, neural network parameters generally lack spatial structure, so we use pure 1D convolutions instead of 2D convolution in our parameter autoencoder and diffusion model.

3. Experiments

In this section, we introduce our experimental setup and then report our results and ablation analysis.

3.1. Setup

Datasets and architectures. We evaluate p-diff across a wide range of datasets, including CIFAR-10/100 [35], STL-10 [9], Flowers [48], Pets [49], Food-101 [4], and ImageNet-1K [10]. To evaluate p-diff's ability to generate new subsets of high-performing network parameters, we conduct experiments on ResNet-18/50 [22], ViT-Tiny/Base [15], ConvNeXt-Tiny/Base [40]. Furthermore, to demonstrate our method's capability to generate full neural network parameters, we also experiment on 5 small hand-designed models: ConvNet-mini, MLP-mini, ResNet-mini, ViT-mini, and ConvNeXt-mini. The architectures of these models are detailed in the Appendix.

Training details. The parameter autoencoder and latent diffusion model both use encoder-decoder architectures with five 1-dimensional CNN layers and one fully-connected layer. For the ImageNet-1K dataset, we take the *pretrained* model from the timm library [71]; for other smaller datasets, we train all model parameters on each dataset until accuracy converges. Then, we fine-tune the last two normalization layers of the model for 300 training steps, saving one check-

	CIFAR-100			STL-10			Flowers			Pets			Food-101			ImageNet-1K		
	orig.	ensem.	p-diff	orig.	ensem.	p-diff	orig.	ensem.	p-diff	orig.	ensem.	p-diff	orig.	ensem.	p-diff	orig.	ensem.	p-diff
ResNet-18	77.4	77.7	**77.9**	95.1	95.1	**95.2**	**80.1**	79.2	79.7	89.0	88.8	**89.1**	79.1	79.4	79.4	69.5	70.3	**70.4**
ResNet-50	78.4	78.6	**78.7**	97.0	96.9	**97.1**	85.9	85.8	**86.1**	**92.6**	92.1	92.5	**80.4**	80.0	80.1	78.3	78.5	78.5
ViT-Tiny	86.7	86.4	86.7	**97.8**	97.6	97.7	87.6	87.5	87.6	**90.5**	89.9	90.4	87.4	87.4	**87.5**	75.2	75.4	75.4
ViT-Base	91.2	91.1	91.2	99.1	99.1	99.1	98.1	98.0	**98.2**	94.1	94.1	**94.4**	**91.3**	91.2	91.2	85.1	85.1	85.1
ConvNeXt-Tiny	89.8	89.6	89.8	**99.1**	98.9	99.0	98.2	98.2	**98.3**	**94.2**	93.7	93.9	91.6	91.5	91.6	83.4	83.7	83.7
ConvNeXt-Base	93.2	93.1	93.2	99.7	99.6	99.7	99.5	99.5	99.5	**95.2**	94.7	95.0	92.8	92.9	**93.0**	85.1	85.3	85.3

Table 1. Partial parameters generated by p-diff match or exceed original models' performance. **Bold entries** are best results.

	ConvNet-mini			MLP-mini			ResNet-mini			ViT-mini			ConvNeXt-mini		
	original	ensemble	p-diff	original	ensemble	p-diff	original	ensemble	p-diff	original	ensemble	p-diff	original	ensemble	p-diff
CIFAR-10	**56.0**	55.6	55.8	41.8	41.8	**42.4**	**58.6**	58.1	58.5	73.4	73.6	73.6	**72.0**	71.7	71.9
STL-10	**46.4**	45.6	46.1	35.0	35.0	**35.2**	51.5	51.4	**51.8**	52.3	52.0	**52.6**	48.1	47.8	48.1

Table 2. The best validation accuracy comparison between original models and full parameters generated by p-diff. P-diff demonstrates strong generality across five architectures (ConvNet-mini, MLP-mini, ResNet-mini, ViT-mini, and ConvNeXt-mini) with number of parameters ranging from 24K to 81K. Each architecture has a distinct structural design. **Bold entries** indicate best results.

point at each step as the original models for p-diff training. For our hand-designed architectures, we also first train the models from scratch and then follow the same fine-tuning procedures. Note that during the fine-tuning procedures, we fine-tune all the model parameters rather than a subset of the model. The noise scales (σ_v and σ_z) for the Gaussian noise added to the parameters and latent representations are set to 0.001 and 0.1, respectively. In most cases, data collection and training for p-diff can be completed within 1 to 3 hours using a single NVIDIA A100 GPU with 40GB of memory.

Inference details. We synthesize 200 sets of novel parameters by feeding random noise into the trained diffusion model and decoder. These synthesized parameters are then concatenated with the aforementioned remaining parameters to form the generated models. By default, p-diff is compared to the baselines using *the best validation accuracy*.

Baselines. i) The best validation accuracy of the original models is denoted as 'original'. ii) Average weight ensemble [37, 73] of the original models is denoted as 'ensemble'.

3.2. Results

Performance on partial parameter generation. Table 1 compares the performance of the generated models with the two baselines across 7 datasets and 6 architectures. We make several notable observations: i) In most cases, our method achieves similar or better results compared to both baselines. This demonstrates that our method can efficiently learn the distribution of high-performing parameters and generate superior models from random noise. ii) Our method consistently performs well across all datasets and architectures, highlighting its strong generality.

Generalization to full parameter generation. Until now, we have evaluated the effectiveness of our approach in synthesizing a subset of model parameters, *i.e.,* batch normalization parameters. *What about synthesizing entire model parameters?* We validate our approach's capability to generate full neural network parameters across 5 small, hand-designed architectures (ranging from 24K to 81K parameters due to GPU memory constraints): ConvNet-mini, MLP-mini, ResNet-mini, ViT-mini, and ConvNeXt-mini.

We evaluate full parameter synthesis with the five hand-designed architectures on CIFAR-10 and STL-10 datasets. The results are presented in Table 2. It is worth noting that all architectures include unique structures. For example, ResNet-mini includes skip connections, ViT-mini contains self-attention mechanisms, and ConvNeXt-mini has depth-wise separable convolutions. The results demonstrate that our synthesized models achieve comparable or even better performance than the original ones across all architectures.

3.3. Ablation Analysis

In this section, we conduct extensive ablation studies to demonstrate our method's characteristics in different data collection and training settings. We focus our analysis on ResNet-18 and CIFAR-100, and report both the best accuracy and the average accuracy across all generated models.

Noise augmentation. In Section 2.3, we propose noise augmentation during training to enhance the robustness and generalization of the parameter autoencoder. Here, we ablate the effectiveness of applying the augmentations to the input parameters and latent representations, respectively. Results in Table 3a show that: i) Noise augmentation is important for generating high-performing neural network parameters. ii) Applying noise augmentation to the latent representations yields greater performance gains than applying it to the input parameters. iii) Our default setting, which jointly applies noise augmentation to both input parameters

noise augmentation	best	average
none	77.5	76.7
parameter	77.4	76.7
latent	77.9	77.6
both	77.9	**77.7**

(a) **Noise augmentation.** Using noise augmentation is important for p-diff to generate model parameters with high performance.

noise scale	best	average
0.001	77.9	**77.7**
$\times$ 0.01	77.9	77.6
$\times$ 5	77.9	77.6
$\times$ 2000	77.9	77.6
$\times$ 5000	77.3	76.8

(b) **Parameter Noise.** Generated models' performance is less sensitive to parameter noise.

noise scale	best	average
0.1	77.9	**77.7**
$\times$ 0.01	77.4	76.7
$\times$ 0.1	77.4	76.8
$\times$ 10	77.9	77.6
$\times$ 100	77.5	77.2

(c) **Latent noise.** Generated models' performance is more sensitive to latent noise.

optimizer	original	best	average
SGD	77.6	77.5	77.3
Adam [30]	79.7	79.6	79.4
AdamW [42]	77.4	77.9	77.7

(d) **Optimizer.** For original models trained with different optimizers, p-diff can generate similar or better performing parameters.

samples (K)	best	average
10	77.0	76.6
50	77.5	77.2
200	77.8	77.5
300	77.9	**77.7**
400	**78.0**	77.6

(e) **Number of pretrained model samples.** A larger K can improve the performance of generated model parameters.

layer	orginal	best	average
conv. layer (1)	77.8	77.8	77.3
fc layer	77.8	77.8	77.7
BN layers (10-13)	77.4	77.5	77.0
BN layers (14-15)	77.7	77.3	76.8
BN layers (16-17)	77.4	77.9	77.7

(f) **Subsets of model parameters**. P-diff consistently generates similar or better performing parameters across parameter groups.

Table 3. **P-diff ablation experiments.** We present the best validation accuracy of the original models and the best and the average accuracies of the generated models. Unless stated otherwise, p-diff is trained on $K = 300$ samples of BN parameters at the 16th and 17th layers of ResNet-18, with parameter and latent noise augmentation applied. Defaults are marked in gray. **Bold entries** are best results.

and latent representations, achieves the best results.

We further evaluate the sensitivity of model performance to the scale of the noise applied to input parameters and latent representations. To do this, we vary one noise scale (σ_v or σ_z) while keeping the other constant. The resulting accuracies in Table 3b and Table 3c show that generated models are more sensitive to latent noise. Also, the current set of noise scales achieves the optimal performance.

Generalization to other optimizers. Our default setting employs the AdamW [42] optimizer for training the original models. To validate our method's robustness across different optimization algorithms, we conduct experiments using SGD and Adam [30], with other experimental protocols detailed in the previous section remain the same. The results in Table 3d demonstrate that our approach achieves comparable performance to the original models across all three optimizers, confirming its generality.

The number of training models. Table 3e varies the size of p-diff's training data (K), *i.e.*, the number of saved original model checkpoints. We observe that both the best accuracy and the average accuracy consistently improve as the number of original models increases, gradually converging as K reaches 400. This indicates the feasibility of our method across different training data sizes. We note that the model performance with a small number of original models (K = 10) is lower than those with larger sizes of training data. This is expected, since it may be very difficult for the diffusion model to learn the target distribution effectively when only a few samples are provided during training.

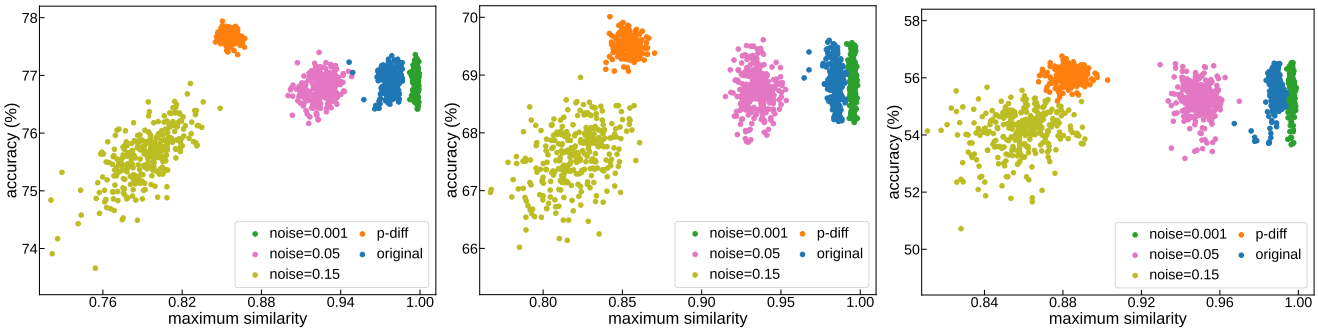
Where to apply p-diff. We default to synthesizing the parameters of the last two batch normalization layers in ResNet-18. To investigate the robustness of our approach in generating other parameters, we also conduct experiments on batch normalization layers at different depths, as well as other types of layers. As shown in Table 3f, our approach consistently generates models that perform on par with the original ones across BN layers at all depths. Additionally, generated parameters on both convolutional and fully-connected layers in our approach achieve reasonable performance compared with the ones in original models.

4. Is P-diff Only Memorizing?

We have demonstrated above that p-diff is able to generate models with high performance. Nevertheless, this is meaningful only if these generated models are quite different from the training ones, while still capturing the distribution of high-performing model checkpoints.

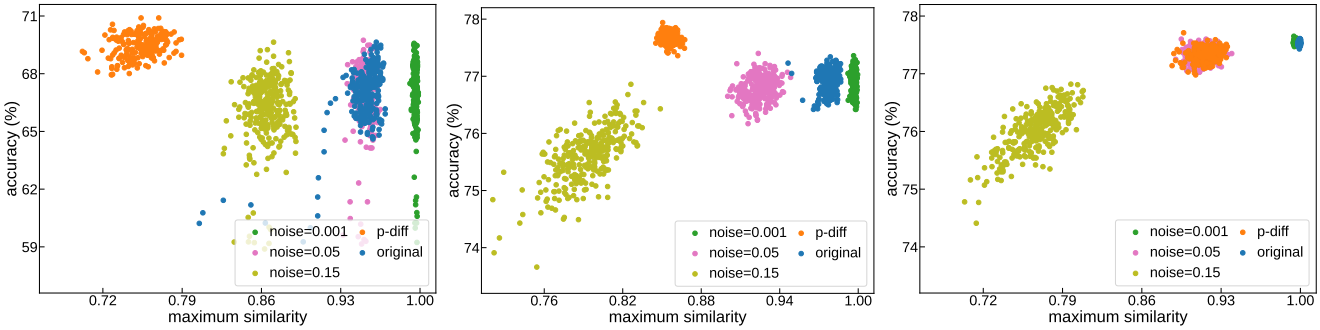
In this section, we investigate the difference among the original, noise-added, and generated models. We propose a similarity metric for model checkpoints to quantitatively assess the novelty of models generated by p-diff.

Questions and experiment designs. Here, we first ask the following questions: 1) Does p-diff merely memorize the original model checkpoints in the training set? 2) Are generated models equivalent to the original models with Gaussian noise added? For p-diff to truly capture the distribution of high-performing checkpoints, it should generate novel parameters that perform differently from the original models. To verify this, we examine and visualize the differences in model predictions among original, noise-added, and generated checkpoints. We carry out our analysis on ResNet-18 [22] and CIFAR-100 [35] under the default setting.



(a) epoch = 10 (default) (b) epoch = 3 (c) epoch = 1

Figure 3. **p-diff generalizes to under-trained original models.** For original models trained for 1 or 3 epochs before the fine-tuning steps, the generated models can still achieve high accuracy with low similarity to original models. Results are on ResNet-18 and CIFAR-100.



(a) learning rate = 0.3 (b) learning rate = 0.03 (default) (c) learning rate = 0.0003

Figure 4. **Diversity of original models is important for the novelty of generated models.** We use learning rates of 0.03, 0.003, and 0.3 for fine-tuning and saving the normalization layers of the converged model as training samples. Higher learning rate leads to greater original model diversity and lower maximum similarity of generated models. Results are on ResNet-18 and the CIFAR-100 dataset.

Similarity metric. While Euclidean and cosine distances provide parameter-level comparisons, they cannot account for the difference in parameter value range across parameter groups. Most importantly, they fail to capture the actual behavioral differences between models. Therefore, we measure the similarity between a pair of models by calculating the Intersection over Union (IoU) of their wrong predictions. The IoU is formulated as follows,

$$\text{IoU} = |P_1^{\text{wrong}} \cap P_2^{\text{wrong}}| / |P_1^{\text{wrong}} \cup P_2^{\text{wrong}}|, \quad (7)$$

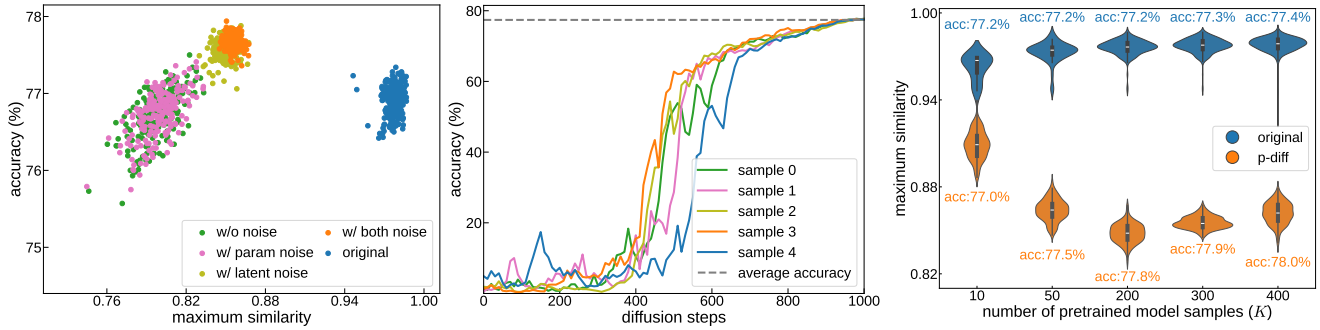
where $P_\cdot^{\text{wrong}}$ denotes the indices of wrong predictions on the validation set, $\cap$ and $\cup$ represent union and intersection operations of sets. A lower IoU indicates a lower similarity between the predictions of the two models and is desired. From now on, we use the IoU of wrong predictions as the similarity metric throughout our paper.

Measure novelty with maximum similarity. For each model checkpoint, we calculate its similarity to every original model and take the maximum similarity as a measure of the model checkpoint's novelty. We compute the maximum similarity of each original, noise-added, and generated model. Note that when evaluating the maximum similarity of an original model, we exclude its similarity to itself and only consider the similarity to other original models.

To compare p-diff generated models with Gaussian noise-added models, we evaluate the latter at noise scales of 0.001, 0.05, and 0.15. In Figure 4a, we plot validation accuracy against maximum similarity for the original (blue), noise-added, and p-diff generated models (orange). As the noise scale increases, the accuracy and maximum similarity of the noise-added models decrease simultaneously. In contrast, p-diff's generated models are all located in the upper-left region, indicating both high accuracy and low maximum similarity. This suggests that our approach achieves an effective trade-off between accuracy and novelty.

Under-trained original models. In the default setting, we use a well-trained model and save its checkpoints from 300 additional fine-tuning steps as training samples for p-diff. Here, we explore whether a model trained for a shorter duration can still enable p-diff to generate novel parameters with reasonable performance compared to the original ones.

Concretely, we train models for only 1 or 3 epochs, fine-tune the selected subset of parameters, and then save checkpoints. The accuracy and maximum similarity of the original, generated, and noise-added models are plotted in Figure 3. As expected, the accuracy of these original models is



(a) noise augmentations. (b) accuracy vs diffusion steps. (c) number of training samples.

Figure 5. (a) shows the impact of parameter and latent noise augmentation on the novelty and accuracy of generated models. Using both parameter and latent noise augmentation achieves the highest accuracy. (b) illustrates the accuracy trajectories across different diffusion steps during inference. Our approach generates high-performing parameters through diverse paths. (c) presents the distributions of maximum similarity, where thickness indicates density. A large set of original models leads to higher novelty in generated models.

much lower due to shorter training. However, in all cases, p-diff can still achieve better trade-off between accuracy and similarity than noise addition to the original models.

Original model diversity. p-diff is designed to capture the underlying distribution of trained high-performing neural network parameters. Therefore, it is important to understand how the diversity of the original models impact the performance and novelty of the generated models. We control the diversity of original models by varying the learning rate during the fine-tuning steps, where we save the original models. Specifically, we experiment with a smaller learning rate (0.0003) and a larger learning rate (0.3), in addition to our default rate (0.03). The results are presented in Figure 4.

We observe that the diversity among original models, measured by maximum similarity, indeed increases with higher fine-tuning learning rate. When training p-diff on model checkpoints fine-tuned with a small learning rate (0.0003), the generated models from p-diff are very similar to the original ones. More importantly, they exhibit a trade-off between validation accuracy and maximum similarity that is similar to that of simple noise addition. However, as the fine-tuning learning rate increases and the diversity of original models grows, p-diff, trained on these more diverse models, shows a clear advantage over noise addition. This highlights the importance of original model's diversity in enhancing the novelty of p-diff's generated models.

Impact of noise augmentation. While Table 3a examines noise augmentation's impact on accuracy, here we take a closer look at its impact on generated model's similarity to original models. In Figure 5a, we plot the accuracy against maximum similarity of the generated models when one or both of the noise augmentations are not used. Augmentation with both latent and input parameter noise enhances model performance while decreasing the maximum similarity.

Additionally, we vary the noise scale and evaluate its impact on the maximum similarity range and best accuracy of p-diff generated models. The results show that our default noise augmentation scale obtains the best trade-off between novelty and accuracy. A small noise augmentation scale reduces the robustness of the parameter autoencoder, resulting in degraded accuracy. Conversely, while larger noise augmentation scales can improve model performance, they negatively impact the diversity of the generated models.

para & latent noise scales	best	similarity range
0.001 & 0.1 (default)	77.9	$0.85 \sim 0.87$
$\times$ 0.01	77.4	$0.75 \sim 0.84$
$\times$ 0.1	77.4	$0.75 \sim 0.83$
$\times$ 10	77.9	$0.82 \sim 0.87$
$\times$ 100	77.6	$0.79 \sim 0.83$

Table 4. **Different noise scales.** Larger noise scales can lead to higher accuracy and higher maximum similarity range.

Parameter trajectories of the p-diff process. To investigate the effectiveness of our method, p-diff, we visualize the accuracy trajectories across different timesteps during the inference stage. Specifically, we first sample five different random noises as the inputs to the diffusion model, and then save the intermediate latent vectors. After that, the intermediate latent vectors are fed into the trained decoder to synthesize the models. Finally, we evaluate the generated models and visualize their accuracy with respect to the number of diffusion steps in Figure 5b.

All trajectories eventually converge to accuracy higher than the average original model accuracy of 77.4%. Importantly, The diverse accuracy trajectories across different random seeds suggest that our method can synthesize model parameters through distinct paths in the parameter space.

From memorization to novelty. A diffusion model can better capture the underlying distribution of training data when a larger number of data points are sampled from that

distribution. To examine how the number of original models (K) affects the novelty of generated models in p-diff, we vary K and visualize the distribution of maximum similarities of all original and generated models with respect to the original models in Figure 5c. Additionally, we report the mean accuracy for both original and generated models.

Across all values of K, p-diff's generated models maintain comparable performance to the original models. When $K = 10$, the generated models exhibit high similarities, all concentrated in a narrow range. With a lower K, p-diff is primarily memorizing the original models. However, as K increases, the maximum similarity decreases, suggesting that p-diff benefits from the increased training checkpoint diversity to generate distinct yet effective parameters. The similarity and accuracy stabilize at $K = 300$ and improve marginally when p-diff is provided with 400 checkpoints.

5. Related Work

Diffusion models. Diffusion models have achieved remarkable success in visual generation. These methods [13, 25–27, 38, 50] are based on non-equilibrium thermodynamics [29, 64]. Their development trajectory is similar to GANs [6, 28, 77], VAEs [31, 53], and flow-based models [14, 55], expanding from unconditional generation to generation conditioned on text, images, or other structured information. Research on diffusion models can be categorized into three main branches. The first branch focuses on enhancing the synthesis quality of diffusion models, such as DALL·E 2 [52], Imagen [59], and Stable Diffusion [57]. The second branch aims to improve the sampling speed, including DDIM [66], Analytic-DPM [2], and DPM-Solver [43]. The final branch involves reevaluating diffusion models from a continuous perspective [17, 67].

Parameter generation. HyperNet [21] dynamically generates the weights of a model with variable architecture. SMASH [5] introduces a flexible scheme based on memory read-writes that can define a diverse range of architectures. G.pt [51] trains a transformer-based diffusion model conditioned on current parameters and target loss or error to generate new parameters. However, the generated models tend to underperform the training model checkpoints. HyperRepresentations [61, 61, 62] uses an autoencoder to capture the latent distribution of trained models. These works focus primarily on the analysis of the parameter space rather than generating high-performing parameters.

Parameter space learning. MetaDiff [74] introduces a diffusion-based meta-learning method for few-shot learning, where a layer is replaced by a diffusion U-Net [58]. Diffusion-SDF [8] proposes a diffusion model for shape completion, single-view reconstruction, and reconstruction of real-scanned point clouds. HyperDiffusion [16] uses a diffusion model on MLPs to generate new neural implicit fields (a representation for 3D shape). DWSNets [45] introduces a symmetry-based approach for designing neural architectures that operate in deep weight spaces. NeRN [1] shows that neural representations can be used to directly represent the weights of a pre-trained convolutional neural network. Different from them, our method is designed to learn high-performing parameter distributions.

Stochastic and Bayesian neural networks. Our approach could be viewed as learning a prior over network parameters, represented by the trained diffusion model. Learning parameter priors for neural networks has been studied in classical literature. Stochastic neural networks (SNNs) [44, 60, 65, 72] also learn such priors by introducing randomness to improve the robustness and generalization of neural networks. The Bayesian neural networks [19, 31, 32, 46, 56] aim to model a probability distribution over neural networks to mitigate overfitting, learn from small datasets, and estimate the uncertainty of model predictions. [20] proposes an easily implementable stochastic variational method as a practical approximation to Bayesian inference for neural networks. They introduce a heuristic pruner to reduce the number of network weights, resulting in improved generalization. [70] combines Langevin dynamics with SGD to incorporate a Gaussian prior into the gradient. This transforms SGD optimization into a sampling process. *Bayes by Backprop* [3] learns a probability distribution prior over the weights of a neural network. These methods mostly operate in small-scale settings, while p-diff shows its effectiveness and potential in real-world neural network architectures.

6. Discussion and Conclusion

Neural networks have several conventional learning paradigms, such as supervised learning [15, 22, 36, 63] and self-supervised learning [7, 12, 23, 24]. In this study, we demonstrate that diffusion models can also be employed to generate high-performing neural network parameters across tasks, network architectures, and training settings. Notably, we demonstrate that our method can generate novel models with distinct behaviors compared to the training checkpoints. Using diffusion process for neural network parameter updates is a potential novel paradigm in deep learning.

We acknowledge that images / videos and parameters are signals of different natures, and this distinction must be handled with care. While diffusion models excel in image / video generation, their applications to model parameters remain relatively underexplored. This gap presents a series of challenges for neural network parameter diffusion. Our approach marks an initial step toward address some of these challenges. Nevertheless, there are still unresolved challenges, including memory constraints for generating the full parameters of large architectures, the efficiency of structure designs, and the stability of performance.

Acknowledgments. We thank Kaiming He, Dianbo Liu, Mingjia Shi, Zheng Zhu, Bo Zhao, Jiawei Liu, Yong Liu, Ziheng Qin, Zangwei Zheng, Yifan Zhang, Xiangyu Peng, Hongyan Chang, Dave Zhenyu Chen, Ahmad Sajedi, and George Cazenavette for valuable discussions and feedbacks. This research is supported by the National Research Foundation, Singapore under its AI Singapore Programme (AISG Award No: AISG2-PhD-2021-08-008).

References

[1] Maor Ashkenazi, Zohar Rimon, Ron Vainshtein, Shir Levi, Elad Richardson, Pinchas Mintz, and Eran Treister. NeRN: Learning neural representations for neural networks. In *ICLR*, 2023. 8

[2] Fan Bao, Chongxuan Li, Jun Zhu, and Bo Zhang. Analytic-DPM: an analytic estimate of the optimal reverse variance in diffusion probabilistic models. In *ICLR*, 2022. 8

[3] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural network. In *ICML*, 2015. 8

[4] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101–mining discriminative components with random forests. In *ECCV*, 2014. 3

[5] Andrew Brock, Theo Lim, J.M. Ritchie, and Nick Weston. SMASH: One-shot model architecture search through hypernetworks. In *ICLR*, 2018. 8

[6] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale gan training for high fidelity natural image synthesis. In *ICLR*, 2019. 8

[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *NeurIPS*, 2020. 8

[8] Gene Chou, Yuval Bahat, and Felix Heide. Diffusion-sdf: Conditional generative modeling of signed distance functions. In *ICCV*, 2023. 8

[9] Adam Coates, Andrew Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In *AISTATS*, 2011. 3

[10] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009. 3

[11] Misha Denil, Babak Shakibi, Laurent Dinh, Marc'Aurelio Ranzato, and Nando De Freitas. Predicting parameters in deep learning. In *NeurIPS*, 2013. 14

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019. 8

[13] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. In *NeurIPS*, 2021. 1, 8

[14] Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation. In *ICLR*, 2019. 8

[15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 3, 8

[16] Ziya Erkoç, Fangchang Ma, Qi Shan, Matthias Nießner, and Angela Dai. Hyperdiffusion: Generating implicit neural fields with weight-space diffusion. In *ICCV*, 2023. 8

[17] Berthy T Feng, Jamie Smith, Michael Rubinstein, Huiwen Chang, Katherine L Bouman, and William T Freeman. Score-based diffusion models as principled priors for inverse imaging. In *ICCV*, 2023. 8

[18] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*, 2017. 14

[19] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *ICML*, 2016. 8

[20] Alex Graves. Practical variational inference for neural networks. In *NeurIPS*, 2011. 8

[21] David Ha, Andrew M. Dai, and Quoc V. Le. Hypernetworks. In *ICLR*, 2017. 8

[22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 3, 5, 8

[23] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *CVPR*, 2020. 8

[24] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, 2022. 8

[25] Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-or. Prompt-to-prompt image editing with cross-attention control. In *ICLR*, 2023. 8

[26] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020. 1, 3, 13

[27] Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P Kingma, Ben Poole, Mohammad Norouzi, David J Fleet, et al. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022. 8

[28] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros. Image-to-image translation with conditional adversarial networks. In *CVPR*, 2017. 1, 8

[29] Christopher Jarzynski. Equilibrium free-energy differences from nonequilibrium measurements: A master-equation approach. *Physical Review E*, 1997. 1, 8

[30] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015. 5

[31] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. In *ICLR*, 2014. 1, 2, 8, 13

[32] Durk P Kingma, Tim Salimans, and Max Welling. Variational dropout and the local reparameterization trick. In *NeurIPS*, 2015. 1, 8

[33] Boris Knyazev, Michal Drozdzal, Graham W Taylor, and Adriana Romero Soriano. Parameter prediction for unseen deep architectures. In *NeurIPS*, 2021. 14

[34] Boris Knyazev, Doha Hwang, and Simon Lacoste-Julien. Can we scale transformers to predict parameters of diverse imagenet models? In *ICML*, 2023. 14

[35] A. Krizhevsky and G. Hinton. Learning multiple layers of features from tiny images. *Master's thesis*, 2009. 3, 5

[36] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *NeurIPS*, 2012. 8

[37] Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In *NeurIPS*, 1994. 4

[38] Alexander C Li, Mihir Prabhudesai, Shivam Duggal, Ellis Brown, and Deepak Pathak. Your diffusion model is secretly a zero-shot classifier. In *ICCV*, 2023. 8

[39] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, 2014. 13

[40] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *CVPR*, 2022. 3

[41] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015. 13

[42] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019. 5

[43] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *NeurIPS*, 2022. 8

[44] Noboru Murata, Shuji Yoshizawa, and Shun-ichi Amari. Network information criterion-determining the number of hidden units for an artificial neural network model. *IEEE transactions on neural networks*, 1994. 1, 8

[45] Aviv Navon, Aviv Shamsian, Idan Achituve, Ethan Fetaya, Gal Chechik, and Haggai Maron. Equivariant architectures for learning in deep weight spaces. In *ICML*, 2023. 8, 14

[46] Radford M Neal. *Bayesian learning for neural networks*. Springer Science & Business Media, 2012. 1, 8

[47] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. In *ICML*, 2022. 1

[48] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *Indian conference on computer vision, graphics & image processing*, 2008. 3

[49] Omkar M Parkhi, Andrea Vedaldi, Andrew Zisserman, and CV Jawahar. Cats and dogs. In *CVPR*, 2012. 3

[50] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *ICCV*, 2023. 8

[51] William Peebles, Ilija Radosavovic, Tim Brooks, Alexei Efros, and Jitendra Malik. Learning to learn with generative models of neural network checkpoints. *arXiv preprint arXiv:2209.12892*, 2022. 8, 14

[52] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 2022. 1, 8

[53] Ali Razavi, Aaron Van den Oord, and Oriol Vinyals. Generating diverse high-fidelity images with vq-vae-2. In *NeurIPS*, 2019. 8

[54] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015. 13

[55] Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *ICML*, 2015. 8

[56] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *ICML*, 2014. 1, 8

[57] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022. 1, 8

[58] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015. 8

[59] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L Denton, Kamyar Ghasemipour, Raphael Gontijo Lopes, Burcu Karagol Ayan, Tim Salimans, et al. Photorealistic text-to-image diffusion models with deep language understanding. In *NeurIPS*, 2022. 1, 8

[60] Wouter F Schmidt, Martin A Kraaijveld, Robert PW Duin, et al. Feed forward neural networks with random weights. In *ICPR*, 1992. 1, 8

[61] Konstantin Schürholt, Boris Knyazev, Xavier Giró-i Nieto, and Damian Borth. Hyper-representations as generative models: Sampling unseen neural network weights. In *NeurIPS*, 2022. 8, 12, 14

[62] Konstantin Schürholt, Michael W Mahoney, and Damian Borth. Towards scalable and versatile weight space learning. *arXiv preprint arXiv:2406.09997*, 2024. 8, 12

[63] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015. 8

[64] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*, 2015. 1, 8

[65] Haim Sompolinsky, Andrea Crisanti, and Hans-Jurgen Sommers. Chaos in random neural networks. *Physical review letters*, 1988. 1, 8

[66] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *ICLR*, 2021. 1, 8

[67] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *NeurIPS*, 2019. 8

[68] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *CVPR*, 2015. 12

[69] Zhi Tian, Chunhua Shen, Hao Chen, and Tong He. Fcos: A simple and strong anchor-free object detector. *IEEE T-PAMI*, 2020. 13

[70] Max Welling and Yee W Teh. Bayesian learning via stochastic gradient langevin dynamics. In *ICML*, 2011. 8

[71] Ross Wightman. GitHub repository: Pytorch image models. *GitHub repository*, 2019. 3

[72] Eugene Wong. Stochastic neural networks. *Algorithmica*, 1991. 1, 8

[73] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *ICML*, 2022. 4

[74] Baoquan Zhang and Demin Yu. Metadiff: Meta-learning with conditional diffusion for few-shot learning. In *AAAI*, 2024. 8, 14

[75] Chris Zhang, Mengye Ren, and Raquel Urtasun. Graph hypernetworks for neural architecture search. *arXiv preprint arXiv:1810.05749*, 2018. 14

[76] Andrey Zhmoginov, Mark Sandler, and Maksym Vladymyrov. Hypertransformer: Model generation for supervised and semi-supervised few-shot learning. In *ICML*, 2022. 14

[77] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *ICCV*, 2017. 1, 8

A. Experimental Settings

In this section, we detail the experimental settings and provide instructions for reproduction.

A.1. Training Recipe

We provide our basic recipes to train p-diff for ResNet-18 and CIFAR-100 in Table 5. For other datasets, the learning rate and training iterations need to be slightly adjusted.

config	value
optimizer	AdamW
learning rate	5e-4
weight decay	5e-4
training epochs	10
optimizer momentum	0.9, 0.999
batch size	128
learning rate schedule	cosine decay
augmentation	`RandomResizedCrop` [68]

(a) training recipe for model checkpoints

config	value
optimizer	AdamW
learning rate	3e-2
weight decay	5e-4
finetuning iterations	200
optimizer momentum	0.9, 0.999
batch size	128
learning rate schedule	cosine decay
augmentation	`RandomResizedCrop` [68]

(b) finetuning recipe for model checkpoints

config	value
optimizer	AdamW
learning rate	2e-5
weight decay	0
training iterations	500
optimizer momentum	0.9, 0.999
batch size	50
learning rate schedule	cosine decay
noise augmentation	σ_v, σ_z=1e-3, 1e-1

(c) training recipe for parameter autoencoder

config	value
optimizer	AdamW
learning rate	1e-4
weight decay	0
training iterations	1000
optimizer momentum	0.9, 0.999
batch size	50
learning rate schedule	cosine decay
diffusion steps	T=1000
noise variance	β_1, β_T=1e-4, 2e-2
noise schedule	linear

(d) training recipe for diffusion model

Table 5. **Training recipes.**

A.2. Hand-designed Architectures

In Section 3.2, we evaluate our method's ability to generate full model parameters by applying it to five small hand-designed architectures: ConvNet-mini, MLP-mini, ResNet-mini, ViT-mini, and ConvNeXt-mini. Here, we use CIFAR-10 as an example and show the details of these architectures.

ConvNet-mini. conv1: (in_channels = 3, out_channels = 8, kernel_size = 7), conv2: (8, 8, 3), conv3: (8, 4, 3), linear1: (in_features = 64, out_features = 16), linear2: (16, 10).

MLP-mini. linear1: (1024, 64), linear2: (64, 16), linear3: (16, 10).

ResNet-mini. conv1: (3, 8, 3), residual block1: (in_channels = 8, out_channels = 8, kernel_size = 3, bottleneck = 4), residual block2: (8, 8, 3, 4), linear1: (128, 16), linear2: (16, 10).

ViT-mini. number tokens: 64, token dimension: 64, depth: 4, number heads: 4, head dimension: 8, feed forward dimension: 64.

ConvNeXt-mini. stage depths: (1, 2, 2, 2), stage dimensions: (16, 32, 32, 32).

A.3. Number of Generated Parameters

In Table 6, we list the number of model parameters to generate for different architectures used in our experiments.

dataset	architecture	generation mode	parameter number
STL-10	ResNet-18	partial	2048
STL-10	ResNet-50	partial	5120
STL-10	ViT-Tiny	partial	768
STL-10	ViT-Base	partial	3072
STL-10	ConvNeXt-Tiny	partial	3072
STL-10	ConvNeXt-Base	partial	4096
STL-10	ConvNet-mini	entire	24262
STL-10	MLP-mini	entire	66970
STL-10	ResNet-mini	entire	25138
STL-10	ViT-mini	entire	80714
STL-10	ConvNeXt-mini	entire	74970

Table 6. Number of parameters to generate for different architectures in our experiments. We take STL-10 as an example.

B. Additional Experiments

B.1. Additional Baselines

We also compare our approach with hyper-representations methods [61, 62] on MNIST and SVHN datasets. To ensure a fair comparison, we use p-diff to generate the same 3-layer convolutional network defined in the hyper-representations papers [61]. As illustrated in Tab. 7, p-diff achieves accuracy similar or better than the original accuracy, whereas models generated using the hyper-representations methods do not perform comparably to the original ones.

dataset	original	S_{KDE30}	SANE_{SUB}	$\text{SANE}_{\text{KDE30}}$	p-diff
MNIST	92.1	68.6	86.7	84.8	**93.3**
SVHN	71.3	54.5	72.3	70.7	**74.1**

Table 7. Model checkpoints generated by p-diff outperform those generated using the hyper-representation methods. Results are in *average accuracy*.

B.2. Is Variational Autoencoder an Alternative to Our Approach?

Variational autoencoder (VAE) [31], as a powerful probabilistic generative model, has demonstrated remarkable success in various generation tasks. Here, we seek to use VAE to generate high-performing parameters. To ensure a fair comparison with p-diff, we implement a vanilla VAE using identical backbone architecture and training iterations. As shown in Table 8, our proposed p-diff generates model parameters with much higher accuracies than VAE.

method	best	average	similarity range
original	77.4	76.9	0.93∼0.98
vae	77.3	76.6	0.74∼0.82
p-diff	**77.9**	**77.7**	0.85∼0.87

Table 8. Comparisons between VAE and p-diff on ResNet-18 and CIFAR-100. Our approach achieves better performances.

B.3. Diffusion Step

We train the diffusion model by default with 1000 diffusion steps. Here, we conduct an ablation study to determine whether fewer steps can still achieve comparable results. As shown in Table 9, we observe that: i) Using too few timesteps can degrade the performance of the generated models. ii) Increasing the diffusion steps beyond 1000 does not provide significant improvement.

diffusion step	best	average
10	74.6	70.5
100	77.9	77.6
1000	77.9	**77.7**
2000	77.8	77.5

Table 9. Effects of diffusion steps on ResNet-18 and CIFAR-100.

B.4. P-diff on Other Vision Tasks

So far, we have only demonstrated p-diff's ability to generate high-performing model weights for image recognition tasks. Below we test our method to generate models for other visual tasks (*i.e.,* object detection, semantic segmentation, and image generation). For each task, we take pretrained models and finetune a subset of parameters in the models to collect 300 model checkpoints. These checkpoints are then used to train p-diff to generate new, high-performing model weights.

Object detection. For this task, we take the pretrained Faster R-CNN [54] and FCOS [69] models, both of which use a ResNet-50 backbone and were trained on the COCO [39] dataset. We report the results in Table 10. Our method generates models with similar or even better performance compared to the original ones.

model	original	p-diff
Faster R-CNN	36.9	**37.0**
FCOS	39.1	39.1

Table 10. P-diff on the object detection task. Results are reported in AP on the COCO dataset.

Semantic segmentation. Here we leverage FCN [41] with a ResNet-50 backbone to evaluate a subset of COCO val2017, on the 20 categories that are present in the Pascal VOC dataset. We generate the parameters of the last normalization layer of ResNet-50 and report the results in Table 11. Our approach can generate high-performing neural network parameters for the semantic segmentation task.

model	original	pdiff
FCN	60.5	**60.7**

Table 11. P-diff on the semantic segmentation task. Results are reported in mIOU on a subset of the COCO dataset.

Image generation. In this task, we use the DDPM [26] model pretrained on the CIFAR-10 dataset. Note that we use p-diff to only generate the last convolutional layer parameters of UNet in DDPM. For evaluation, we compute the FID score of models generated by p-diff and report the result in Table 12. We observe that p-diff achieves competitive performance compared to the original DDPM models, demonstrating its applicability to the image generation task.

model	original	p-diff
DDPM UNet	3.17	**3.19**

Table 12. P-diff on the image generation task. Results are reported in FID on the CIFAR-10 dataset.

B.5. Bottleneck to Scaling Up

Below, we examine how the GPU memory cost of p-diff changes when training with larger model checkpoints. Note that p-diff involves two training stages: autoencoder and diffusion model. For each type of model checkpoint, we increase the latent dimension of the autoencoder based on the size of model checkpoints and record the GPU memory

cost with a fixed batch size of 50. As shown in Table 13, the memory cost of both the autoencoder and the diffusion model increases proportionally with the size of the model checkpoints. Training on model checkpoints with more than 300K parameters usually exceeds the 40GB memory limit of the NVIDIA A100 GPU.

model parameter	AE (MB)	latent dimension	diffusion (MB)
2048	658	128	530
30016	3756	256	1418
100288	13352	512	3594
300288	44882	1024	12942

Table 13. The bottleneck to scaling up is GPU memory.

C. Comparison with Related Studies

We compare our method, p-diff, to each of the following related studies.

G.pt [51] uses the diffusion process as an optimizer (from starting parameter θ to future parameter θ^*). It takes a noised version of the future parameter θ^* and generates a denoised parameter, condition on θ and the current and target loss / error. In contrast, we train p-diff to directly generate high-performing parameters from noise without any conditioning techniques. Also, models generated by G.pt often fall short of the original models' performance. P-diff can match or surpass the accuracy of original models.

MetaDiff [74] uses the diffusion process to perform meta-learning. It takes the features and labels of all training data as inputs and learns to generate target model checkpoints from randomly initialized weights through a denoising process. However, p-diff is designed to generate high-performing neural network weights from random noise.

Hyper-representations [61] introduces a model zoo, containing numerous models trained from scratch. An autoencoder is then trained on the parameters of these models, enabling the generation of new models by sampling from the latent space and decoding the sampled latent vectors. Different from their approach, we use a diffusion model to model the latent distribution of high-performing models and generate novel models from random noise.

HyperTransformer [76] is proposed to generate models for supervised and semi-supervised tasks where labeled data are necessary. It relies on image features to generate small target CNN parameters, while p-diff generates neural network parameters from noise. HyperTransformer is primarily compared with other few-shot learning methods, such as MAML [18]. In contrast, p-diff primarily compares the models it generates with those optimized using AdamW.

GHN methods [33, 34, 75] are originally designed for neural architecture search (NAS) training. These methods predict parameters with better weights through the given architectures as better initializations. Nevertheless, compared to p-diff, they are not able to synthesize model parameters that have comparable results with models trained using AdamW or other optimizers.

Denil *et al.* [11] demonstrates that training only a small fraction (5%) of a neural network's parameters can achieve performance comparable to, or even exceeding, that of the fully trained original model. This suggests that the remaining 95% of the parameters are largely redundant and not critical for the network's training process. In contrast, p-diff focuses on generating high-performing, novel models rather than the redundancy of parameters.

DWSNets [45] introduces an approach to designing neural architectures that operate in deep weight spaces. They focus on what neural architectures can effectively learn and process neural models that are represented as sequences of weights and biases. However, p-diff emphasizes generating neural network parameters rather than designing neural architectures.